

PROJECT REPORT

E-Commerce Order, Return and Refund Management System

Database Management Systems (DBMS) Course Project

Name:	Salman
Roll Number:	25WU0102238
Course:	Database Management Systems (DBMS)
Faculty Name:	[Faculty Name]
Academic Year:	2025 - 2029

Table of Contents

Table of Contents	2
2. Abstract	3
3. Introduction and Problem Statement	4
4. Objectives and Scope	4
5. Software and Hardware Requirements	5
6. ER Diagram	5
7. Relational Schema and Normalization	6
8. Data Dictionary	7
9. SQL Commands Used (DDL and DML)	9
10. Queries with Outputs	11
11. UI Design and Screenshots	13
12. Implementation Details	15
13. Testing	17
14. Conclusion and Future Enhancements	18
15. References	18
16. Appendix: GitHub Repository Link	18

2. Abstract

Abstract

This project presents a database-driven **E-Commerce Order, Return and Refund Management System** built on a centralized MySQL database. E-commerce platforms handle large volumes of interrelated data about customers, products, orders, returns and refunds; when this data is managed manually it becomes duplicated, inconsistent and hard to trace. The system stores all of it in five normalized, related tables and exposes it through a Node.js + Express REST API and a Next.js web interface called *Meridian*.

Customers can register, sign in, browse products with live stock levels, place orders, request returns and follow the status of their refunds. Administrators can manage products and customers and approve or reject returns, issue refunds and mark them completed. Every action in the user interface is executed as an SQL statement through the backend, so the database always reflects the true state of the store.

The report covers the problem statement, objectives, requirements, the ER diagram, the relational schema and normalization, the data dictionary, the SQL commands used, sample queries with outputs, UI screenshots, implementation details, testing, and future enhancements.

3. Introduction and Problem Statement

An e-commerce system manages many kinds of related data: who the customers are, which products exist and how many are in stock, what each customer ordered, which orders were returned and why, and what money was refunded. These records depend on each other, so storing them in separate spreadsheets or paper registers causes data duplication, inconsistency and difficulty in tracking a transaction from purchase to refund.

Problem statement

- E-commerce systems handle large amounts of customer, product, order, return and refund data.
- Managing these records manually leads to duplication, inconsistency and poor traceability.
- Customers need a simple way to view their orders and request returns.
- Administrators need to manage products, customers, orders, returns and refunds efficiently.

A centralized database management system is therefore required to manage the complete e-commerce process. This project uses MySQL as that central store, with constraints and foreign keys guaranteeing that related records stay consistent.

4. Objectives and Scope

Objectives

- Store customer and product information.
- Manage customer orders and product inventory.
- Allow customers to request returns and track return status.
- Manage refunds against approved returns.
- Provide an admin interface to manage the system.
- Maintain consistency between related database records.

Scope

- Customer registration and login.
- Product viewing and inventory management.
- Order management.
- Return requests with approval / rejection.
- Refund processing.
- Admin management and monitoring.
- Database-backed CRUD operations through the UI.

Out of scope: real payment-gateway integration, shipping and courier tracking, and multi-item shopping carts (each order currently references a single product).

5. Software and Hardware Requirements

Software requirements

Component	Tool / Version	Purpose
Operating system	Windows 10/11, Linux or macOS	Development and deployment
Database	MySQL 8.x (MySQL Command Line Client / Workbench)	Data storage and SQL execution
Backend	Node.js 18+ with Express	REST APIs and SQL execution
Frontend	Next.js (React)	Customer and admin user interface
DB driver	mysql2 (Node.js)	Backend-to-MySQL connectivity
Tools	VS Code, Git / GitHub, Postman, web browser	Coding, version control, API testing

Hardware requirements

Component	Minimum
Processor	Intel Core i3 / AMD Ryzen 3 or better
RAM	4 GB (8 GB recommended)
Storage	2 GB free disk space
Display / network	1366×768 or higher; internet for package installation

6. ER Diagram

The ER diagram below shows the five entities of the system and how they relate. A **Customer** **places** many **Orders** (1:N); an **Order** **contains** one **Product** while a Product can appear in many Orders (1:N); an Order may have at most one **Return** (1:0..1); and a Return may generate at most one **Refund** (1:0..1). Primary keys are underlined and foreign keys are marked FK.

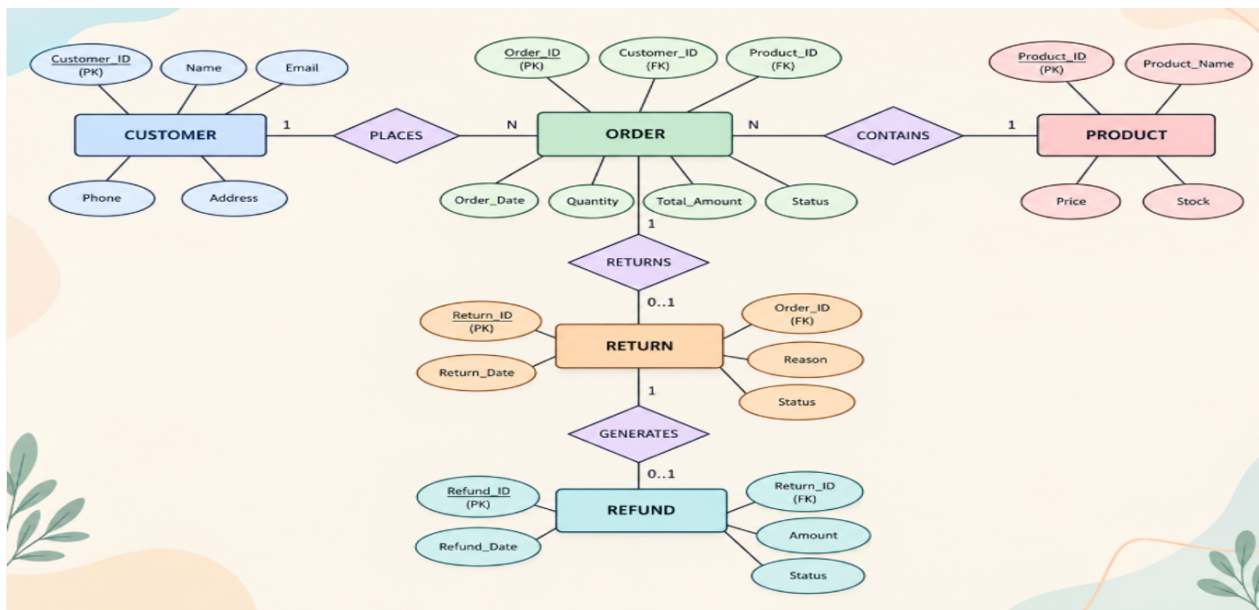


Figure 1: Entity-Relationship diagram of the E-Commerce Order, Return and Refund system

7. Relational Schema and Normalization

Relational schema (derived from the ER diagram)

```
CUSTOMERS (customer_id PK, name, email, phone, address, password_hash)
PRODUCTS  (product_id PK, product_name, price, stock)
ORDERS     (order_id PK, customer_id FK -> CUSTOMERS, product_id FK -> PRODUCTS,
            order_date, quantity, total_amount, status)
RETURNS    (return_id PK, order_id FK -> ORDERS, return_date, reason, status)
REFUNDS    (refund_id PK, return_id FK -> RETURNS, refund_date, amount, status)
```

Mapping of relationships

Relationship	Cardinality	Implemented as
Customer – Places – Order	1 : N	orders.customer_id references customers.customer_id
Product – Contained in – Order	1 : N	orders.product_id references products.product_id
Order – Returns – Return	1 : 0..1	returns.order_id references orders.order_id (UNIQUE)
Return – Generates – Refund	1 : 0..1	refunds.return_id references returns.return_id (UNIQUE)

Normalization

First Normal Form (1NF). Every attribute holds a single atomic value and each table has a primary key. There are no repeating groups; for example, a customer's phone and address are single columns and each order is its own row.

Second Normal Form (2NF). All primary keys are single-column surrogate keys (customer_id, product_id, order_id, return_id, refund_id), so no non-key attribute can depend on only part of a key. All tables are therefore in 2NF.

Third Normal Form (3NF). No non-key attribute depends on another non-key attribute. Product details (name, price, stock) live only in PRODUCTS and customer details only in CUSTOMERS; ORDERS stores just the foreign keys, so a product name is never repeated across orders. Return and refund facts are separated into their own tables because each has its own lifecycle and status. The design is in 3NF.

Design note. ORDERS.total_amount is stored (price × quantity at purchase time) deliberately, so that a later price change in PRODUCTS does not alter the amount of a past order or the amount refunded.

8. Data Dictionary

Table: customers

Stores registered customers.

Column	Data type	Constraints	Description
customer_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique customer identifier
name	VARCHAR(100)	NOT NULL	Customer name
email	VARCHAR(100)	NOT NULL, UNIQUE	Login email
phone	VARCHAR(15)	—	Contact number
address	VARCHAR(255)	—	Delivery address
password_hash	VARCHAR(255)	NOT NULL	Hashed password for login

Table: products

Stores the product catalogue and stock levels.

Column	Data type	Constraints	Description
product_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique product identifier
product_name	VARCHAR(150)	NOT NULL	Product name
price	DECIMAL(10,2)	NOT NULL, CHECK (price >= 0)	Unit price in rupees
stock	INT	NOT NULL, DEFAULT 0, CHECK (stock >= 0)	Units available

Table: orders

One row per customer order.

Column	Data type	Constraints	Description
order_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique order identifier
customer_id	INT	NOT NULL, FOREIGN KEY	Customer who placed the order
product_id	INT	NOT NULL, FOREIGN KEY	Product ordered
order_date	DATE	NOT NULL	Date of the order
quantity	INT	NOT NULL, CHECK (quantity > 0)	Units ordered
total_amount	DECIMAL(10,2)	NOT NULL	price × quantity
status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'	Order status

Table: returns

Return requests raised against orders.

Column	Data type	Constraints	Description
return_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique return identifier
order_id	INT	NOT NULL, UNIQUE, FOREIGN KEY	Order being returned
return_date	DATE	NOT NULL	Date requested
reason	VARCHAR(255)	NOT NULL	Customer's reason
status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'	Pending / Approved / Rejected

Table: refunds

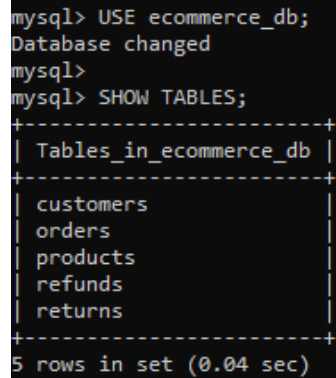
Refunds issued for approved returns.

Column	Data type	Constraints	Description
refund_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique refund identifier
return_id	INT	NOT NULL, UNIQUE, FOREIGN KEY	Return being refunded
refund_date	DATE	NOT NULL	Date refund was created
amount	DECIMAL(10,2)	NOT NULL	Amount refunded
status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'	Pending / Completed

9. SQL Commands Used (DDL and DML)

9.1 Database creation and verification

```
CREATE DATABASE ecommerce_db;  
USE ecommerce_db;  
SHOW TABLES;
```



```
mysql> USE ecommerce_db;  
Database changed  
mysql>  
mysql> SHOW TABLES;  
+-----+  
| Tables_in_ecommerce_db |  
+-----+  
| customers               |  
| orders                  |  
| products                |  
| refunds                 |  
| returns                 |  
+-----+  
5 rows in set (0.04 sec)
```

Figure 2: Output of `USE ecommerce_db` and `SHOW TABLES` — the five tables of the system

9.2 DDL — table creation

```
CREATE TABLE customers (  
  customer_id INT AUTO_INCREMENT PRIMARY KEY,  
  name        VARCHAR(100) NOT NULL,  
  email       VARCHAR(100) NOT NULL UNIQUE,  
  phone       VARCHAR(15),  
  address     VARCHAR(255),  
  password_hash VARCHAR(255) NOT NULL  
);  
CREATE TABLE products (  
  product_id INT AUTO_INCREMENT PRIMARY KEY,  
  product_name VARCHAR(150) NOT NULL,  
  price       DECIMAL(10,2) NOT NULL CHECK (price >= 0),  
  stock       INT NOT NULL DEFAULT 0 CHECK (stock >= 0)  
);  
CREATE TABLE orders (  
  order_id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  product_id INT NOT NULL,  
  order_date DATE NOT NULL,  
  quantity INT NOT NULL CHECK (quantity > 0),  
  total_amount DECIMAL(10,2) NOT NULL,  
  status VARCHAR(20) NOT NULL DEFAULT 'Pending',  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id),  
  FOREIGN KEY (product_id) REFERENCES products(product_id)  
);  
CREATE TABLE returns (  
  return_id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id INT NOT NULL UNIQUE,  
  return_date DATE NOT NULL,  
  reason VARCHAR(255) NOT NULL,  
  status VARCHAR(20) NOT NULL DEFAULT 'Pending',  
  FOREIGN KEY (order_id) REFERENCES orders(order_id)  
);  
CREATE TABLE refunds (  
  refund_id INT AUTO_INCREMENT PRIMARY KEY,  
  return_id INT NOT NULL UNIQUE,  
  refund_date DATE NOT NULL,  
  amount DECIMAL(10,2) NOT NULL,  
  status VARCHAR(20) NOT NULL DEFAULT 'Pending',  
  FOREIGN KEY (return_id) REFERENCES returns(return_id)  
);
```

9.3 DML — insert, update, delete

```
-- Insert a product (Admin: Add product)
INSERT INTO products (product_name, price, stock)
VALUES ('Wireless Mouse', 799.00, 100);

-- Place an order
INSERT INTO orders (customer_id, product_id, order_date, quantity, total_amount)
VALUES (3, 7, CURDATE(), 1, 2199.00);

-- Customer requests a return
INSERT INTO returns (order_id, return_date, reason)
VALUES (5, CURDATE(), 'damage');

-- Admin approves the return, then issues a refund
UPDATE returns SET status = 'Approved' WHERE return_id = 1;
INSERT INTO refunds (return_id, refund_date, amount)
SELECT r.return_id, CURDATE(), o.total_amount
FROM returns r JOIN orders o ON o.order_id = r.order_id
WHERE r.return_id = 1;

-- Admin marks the refund completed
UPDATE refunds SET status = 'Completed' WHERE refund_id = 1;

-- Admin deletes a product (trash icon)
DELETE FROM products WHERE product_id = 101;
```

The sample data produced by these operations is visible in the UI screenshots in Section 11 (for example, order #5 for the Portable Bluetooth Speaker, return #1 with reason "damage", and refund #1 of Rs. 2,199).

10. Queries with Outputs

Q1. First five products in the catalogue

```
SELECT product_id, product_name, price, stock
FROM products
ORDER BY product_id
LIMIT 5;
```

Output

product_id	product_name	price	stock
1	Wireless Bluetooth Headphones	2499.00	50
2	Smart Watch Series 5	3999.00	35
3	Wireless Mouse	799.00	100
4	Mechanical Gaming Keyboard	3499.00	40
5	USB-C Fast Charger	1299.00	75

Q2. Orders with product names (JOIN) — used by the Orders page

```
SELECT o.order_id, p.product_name, o.order_date, o.quantity,
       o.total_amount, o.status
FROM orders o JOIN products p ON p.product_id = o.product_id
ORDER BY o.order_id DESC;
```

Output

order_id	product_name	order_date	quantity	total_amount	status
5	Portable Bluetooth Speaker	2026-10-06	1	2199.00	Pending
4	Laptop Backpack	2026-10-06	1	1899.00	Pending
3	Smart Watch Series 5	2026-10-06	1	3999.00	Pending
2	Wireless Bluetooth Headphones	2026-10-06	1	2499.00	Pending

Q3. Returns with their orders and products

```
SELECT r.return_id, r.order_id, p.product_name, r.reason, r.status
FROM returns r
JOIN orders o ON o.order_id = r.order_id
JOIN products p ON p.product_id = o.product_id;
```

Output

return_id	order_id	product_name	reason	status
1	5	Portable Bluetooth Speaker	damage	Approved

Q4. Refunds with order and product details — used by the Refunds page

```
SELECT f.refund_id, f.return_id, f.amount, f.status,
       o.order_id, p.product_name
FROM refunds f
JOIN returns r ON r.return_id = f.return_id
JOIN orders o ON o.order_id = r.order_id
JOIN products p ON p.product_id = o.product_id;
```

Output

```
+-----+-----+-----+-----+-----+-----+
| refund_id | return_id | amount | status | order_id | product_name |
+-----+-----+-----+-----+-----+-----+
|          1 |          1 | 2199.00 | Pending |          5 | Portable Bluetooth Speaker |
+-----+-----+-----+-----+-----+-----+
```

Q5. Aggregate: number of orders and total value per customer

```
SELECT customer_id, COUNT(*) AS orders_placed, SUM(total_amount) AS total_value
FROM orders
GROUP BY customer_id;
```

Output

```
+-----+-----+-----+
| customer_id | orders_placed | total_value |
+-----+-----+-----+
|          3 |          4 |    10596.00 |
+-----+-----+-----+
```

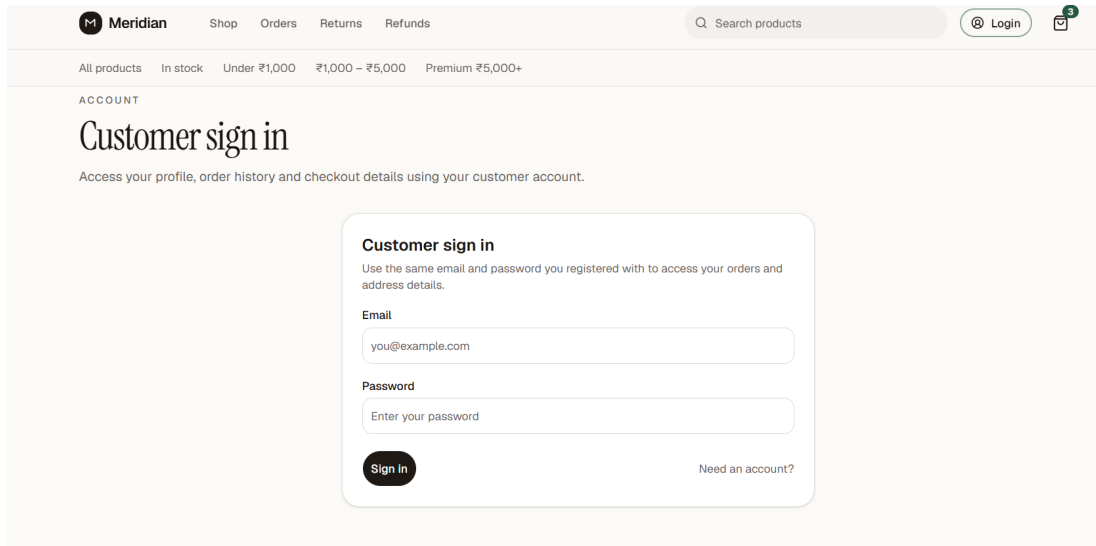
Q6. Query given during Presentation-II

```
-- [Paste the exact query given during Presentation-II here]

-- Output:
-- [Paste the output / screenshot here]
```

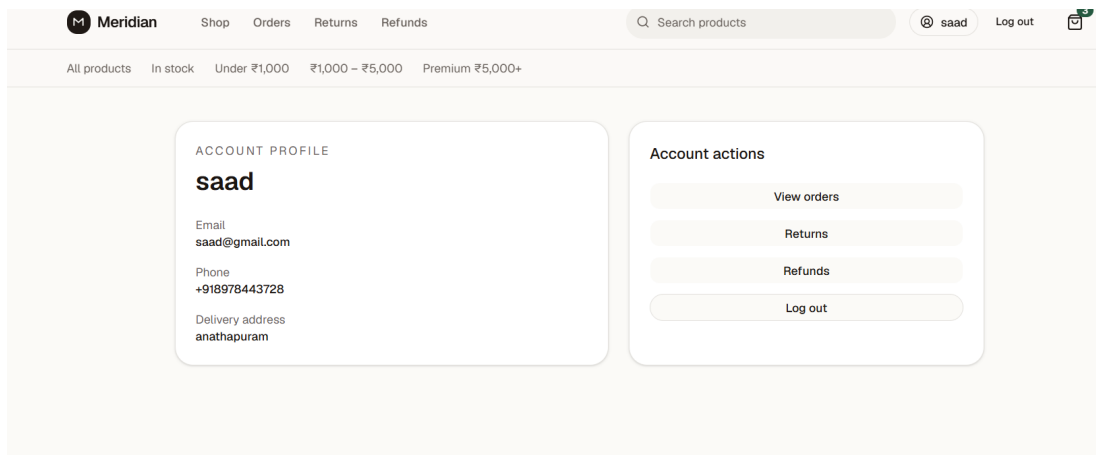
11. UI Design and Screenshots

The user interface, branded **Meridian**, is a Next.js application with a clean, minimal design: a top navigation bar (Shop, Orders, Returns, Refunds), a product search box, a login button and a cart badge. Price filters (Under Rs. 1,000; Rs. 1,000–5,000; Premium Rs. 5,000+) and an in-stock filter sit below the navigation bar.



The screenshot shows the 'Customer sign in' page of the Meridian application. The top navigation bar includes the Meridian logo, links for Shop, Orders, Returns, and Refunds, a search bar, a login button, and a cart icon with a notification badge. Below the navigation bar, there are filters for 'All products', 'In stock', and price ranges: 'Under ₹1,000', '₹1,000 – ₹5,000', and 'Premium ₹5,000+'. The main content area is titled 'ACCOUNT' and 'Customer sign in'. It includes a sub-header 'Access your profile, order history and checkout details using your customer account.' and a sign-in form. The form has fields for 'Email' (with the placeholder 'you@example.com') and 'Password' (with the placeholder 'Enter your password'). A 'Sign in' button is at the bottom left of the form, and a link 'Need an account?' is at the bottom right.

Figure 3: Customer sign-in page (email and password)



The screenshot shows the 'Account profile' page of the Meridian application. The top navigation bar is identical to the previous screenshot, but the login button now displays the name 'saad' and a 'Log out' button is added. Below the navigation bar, the same filters are present. The main content area is titled 'ACCOUNT PROFILE' and 'saad'. It includes a sub-header 'ACCOUNT PROFILE' and a list of account details: 'Email: saad@gmail.com', 'Phone: +918978443728', and 'Delivery address: anathapuram'. To the right of the account details is a section titled 'Account actions' with four buttons: 'View orders', 'Returns', 'Refunds', and 'Log out'.

Figure 4: Account profile with name, email, phone, delivery address and account actions

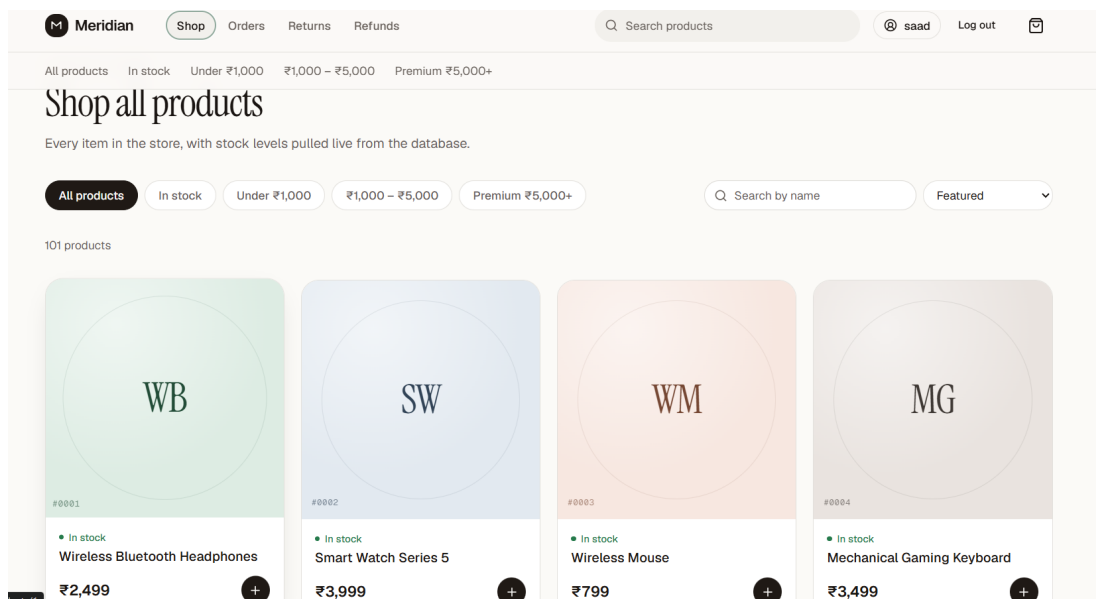


Figure 5: Shop page — 101 products with live stock status and prices pulled from the database

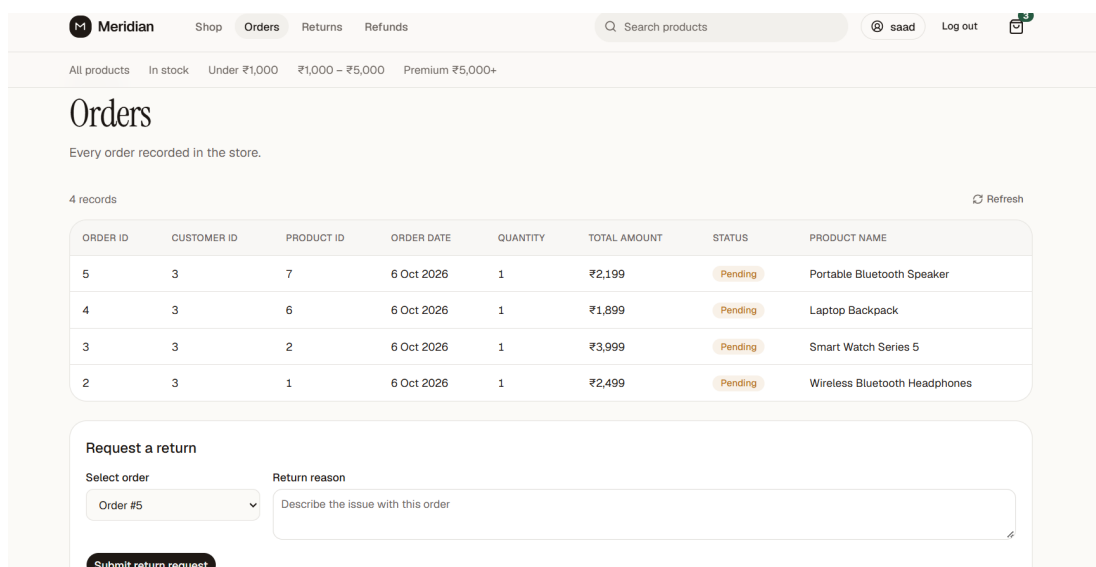


Figure 6: Orders page with the "Request a return" form (select order and give a reason)

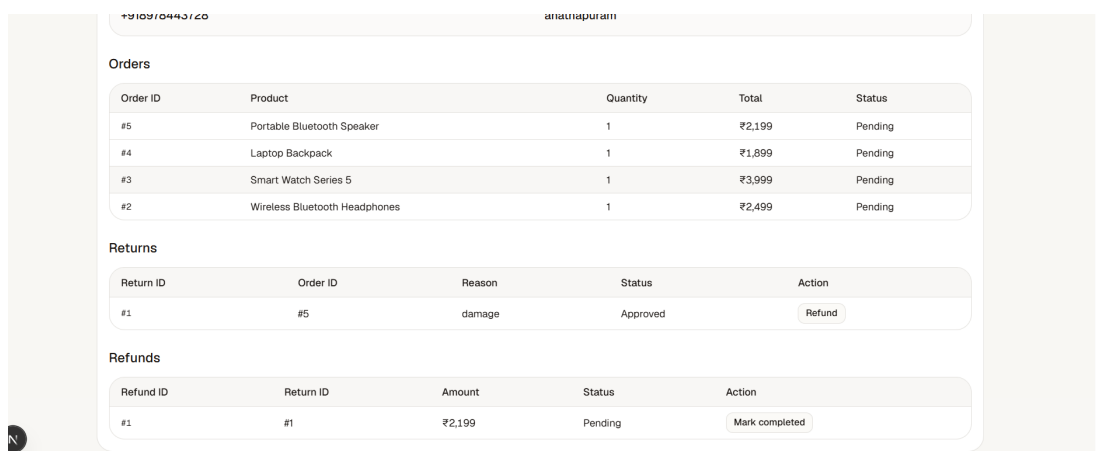


Figure 7: Orders, Returns and Refunds tables with action buttons (Refund, Mark completed)

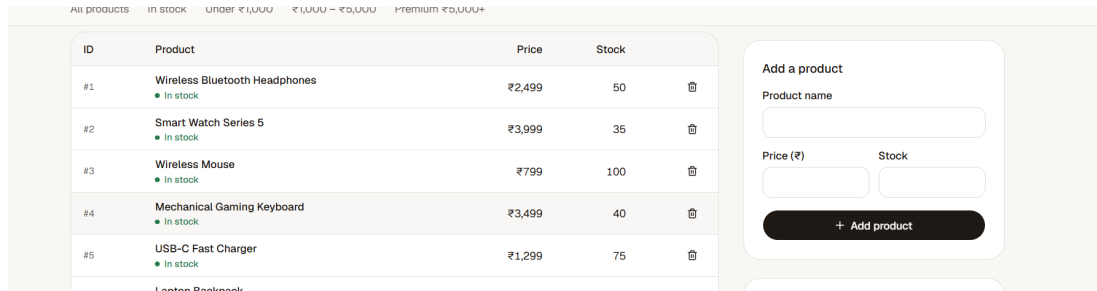


Figure 8: Admin product list with delete action and the "Add a product" form

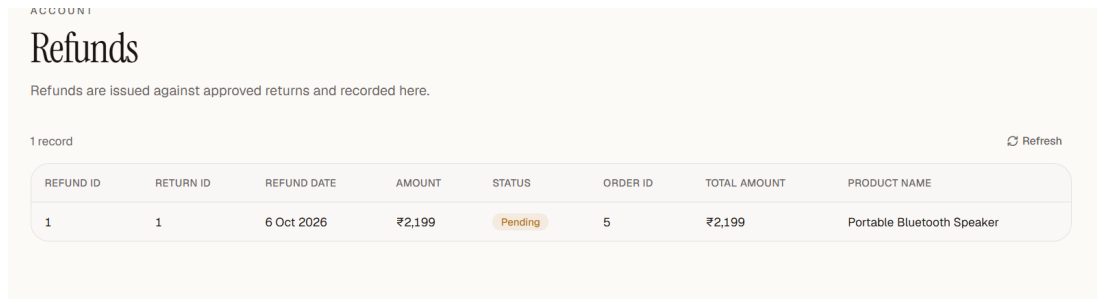


Figure 9: Refunds page showing refund #1 of Rs. 2,199 against order #5

12. Implementation Details

Technology stack and architecture

Next.js UI (Meridian) --HTTP/JSON--> Node.js + Express REST API (routes + validation) --SQL--> MySQL (ecommerce_db)

The frontend never connects directly to MySQL. It sends HTTP requests to the Express backend; the backend runs parameterised SQL queries through a connection pool and returns JSON, which the UI renders. Any change made in the UI is therefore committed to MySQL and is visible on the next refresh.

Layer	Technology	Responsibility
Presentation	Next.js (React)	Pages: sign-in, profile, shop, orders, returns, refunds, admin products
Application	Node.js + Express	REST endpoints, input validation, business rules
Data	MySQL 8 + mysql2	Tables, constraints, joins and persistence

Database connectivity (representative code)

```
// db.js
const mysql = require('mysql2/promise');
const pool = mysql.createPool({
  host: 'localhost', user: 'root', password: process.env.DB_PASSWORD,
  database: 'ecommerce_db', waitForConnections: true, connectionLimit: 10
});
module.exports = pool;
```

Key endpoints (representative code)

```

// GET /api/orders - orders joined with product names
app.get('/api/orders', async (req, res) => {
  const [rows] = await pool.query(
    `SELECT o.*, p.product_name FROM orders o
      JOIN products p ON p.product_id = o.product_id
      ORDER BY o.order_id DESC`);
  res.json(rows);
});

// POST /api/returns - customer requests a return
app.post('/api/returns', async (req, res) => {
  const { order_id, reason } = req.body;
  await pool.query(
    `INSERT INTO returns (order_id, return_date, reason) VALUES (?, CURDATE(), ?)`,
    [order_id, reason]);
  res.status(201).json({ message: 'Return requested' });
});

// POST /api/refunds - admin issues a refund for an approved return
app.post('/api/refunds', async (req, res) => {
  await pool.query(
    `INSERT INTO refunds (return_id, refund_date, amount)
      SELECT r.return_id, CURDATE(), o.total_amount
      FROM returns r JOIN orders o ON o.order_id = r.order_id
      WHERE r.return_id = ? AND r.status = 'Approved`, [req.body.return_id]);
  res.status(201).json({ message: 'Refund created' });
});

```

Data integrity is enforced by the database itself: foreign keys prevent orphan returns and refunds, UNIQUE keys on returns.order_id and refunds.return_id allow at most one return per order and one refund per return, and CHECK constraints stop negative prices, stock and quantities.

13. Testing

Functional testing was carried out through the UI and verified against the MySQL tables. The results below correspond to the screenshots in Section 11.

#	Test case	Input / action	Expected result	Result
T1	Open sign-in page	Navigate to Login	Email and password form is shown	Pass
T2	View profile	Sign in as saad	Name, email, phone and address displayed	Pass
T3	Browse products	Open Shop	101 products listed with stock status and prices	Pass
T4	View orders	Open Orders	Orders #2-#5 listed with product names and totals	Pass
T5	Request a return	Select Order #5, reason "damage", submit	Return #1 created for order #5	Pass
T6	Approve return	Admin approves return #1	Status shows Approved and a Refund button appears	Pass
T7	Issue refund	Click Refund on return #1	Refund #1 of Rs. 2,199 created with status Pending	Pass
T8	Admin product list	Open admin products	Products with price, stock, delete and add form shown	Pass
T9	Duplicate return	Request a second return for order #5	Rejected by UNIQUE constraint on returns.order_id	[verify]
T10	Invalid login	Wrong password	Error message, no access	[verify]

14. Conclusion and Future Enhancements

The project delivers a working database-driven e-commerce system in which customers, products, orders, returns and refunds are stored in related MySQL tables and managed through a modern web interface. The normalized (3NF) schema, foreign keys and constraints keep the data consistent, and the Express API ensures that every UI action is reflected in the database, giving accurate tracking of an order from purchase through return to refund.

Future enhancements

- Multi-item orders and a persistent shopping cart (an `order_items` table).
- Payment-gateway integration and automatic refund to the original payment method.
- Role-based access control with JWT authentication and separate admin login.
- Email / SMS notifications for order, return and refund status changes.
- Automatic stock restoration when a returned item is received.
- Sales and returns analytics dashboard for administrators.
- Database transactions and triggers for stronger consistency.

15. References

- R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, Pearson.
- A. Silberschatz, H. Korth and S. Sudarshan, *Database System Concepts*, McGraw-Hill.
- MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/>
- Express.js documentation — <https://expressjs.com/>
- Next.js documentation — <https://nextjs.org/docs>
- Node.js documentation — <https://nodejs.org/docs>
- mysql2 package — <https://github.com/sidorares/node-mysql2>

16. Appendix: GitHub Repository Link

Project source code: [\[Paste your GitHub repository URL here\]](#)